

CSE-322: End-Semester Examination

Winter-2026

April 25, 2026

Full Marks: 70

Time: 3:00 hours

General Instructions: This is a closed book examination. Please be precise, brief and to the point in your answer. Unnecessary blabbers or unreadable handwriting will fetch deduction of marks.

1. Let B_1 and B_2 be decidable languages and $B_1 \cdot B_2 = \{w \mid w = xy \text{ such that } x \in B_1, y \in B_2\}$. Then prove that $B_1 \cdot B_2$ is decidable.

Solution: Let B_1 and B_2 be decidable languages. By the definition of decidability, there exist Turing machines M_1 and M_2 that are **deciders** such that $L(M_1) = B_1$ and $L(M_2) = B_2$. We construct a new Turing machine M to decide $B_1 \cdot B_2$.

On input string w of length n :

- (a) **Loop over all decompositions:** For each of the $n + 1$ possible ways to divide w into two substrings x and y such that $w = xy$ (including empty strings ε):
- Run M_1 on input x .
 - Run M_2 on input y .
- (b) **Acceptance condition:** If both M_1 accepts x and M_2 accepts y , then M halts and **accepts**.
- (c) **Rejection condition:** If the loop finishes checking all $n + 1$ decompositions and no such split resulted in both machines accepting, M halts and **rejects**.

We must show that M is a decider and that $L(M) = B_1 \cdot B_2$.

- M is a decider: For any input w of length n , there are exactly $n + 1$ finite decompositions of w . Because M_1 and M_2 are deciders, they are guaranteed to halt on all inputs x and y in finite time. Since M performs a finite number of steps ($n + 1$ iterations) and each step takes a finite amount of time, M is guaranteed to halt on all inputs. Therefore, M is a decider.
- If M accepts w , then $w \in B_1 \cdot B_2$: Assume M accepts w . According to the algorithm, this only happens if there exists at least one specific decomposition $w = xy$ where M_1 accepts x and M_2 accepts y . Since $L(M_1) = B_1$ and $L(M_2) = B_2$, it follows that $x \in B_1$ and $y \in B_2$. Thus, $w = xy \in B_1 \cdot B_2$.
- If $w \in B_1 \cdot B_2$, then M accepts w : Assume $w \in B_1 \cdot B_2$. By definition, there exists some split $w = xy$ such that $x \in B_1$ and $y \in B_2$. Because M systematically tests every possible decomposition of w , it will eventually test this specific $w = xy$. Since $x \in B_1$, decider M_1 will accept x . Since $y \in B_2$, decider M_2 will accept y . Consequently, M will accept w .

Since M halts on all inputs and correctly accepts strings if and only if they belong to $B_1 \cdot B_2$, the language $B_1 \cdot B_2$ is decidable. ■

Proposed Marking Scheme:

- Stating that deciders for B_1 and B_2 exist: 2 Mark
- Constructing the Turing machine M : 6 Marks

2. Consider the following CFG $G = (V, \Sigma, P, S)$ with start symbol S as follows.

- $S \rightarrow aaSbbb \mid \varepsilon$

What is $L(G)$? Formally, prove your answer.

Solution: To prove that $L(G) = \{a^{2n}b^{3n} \mid n \geq 0\}$,

Part 1: $L(G) \subseteq \{a^{2n}b^{3n} \mid n \geq 0\}$

We proceed by mathematical induction on the number of derivation steps, k , to show that if $S \Rightarrow^k w$ (where $w \in \Sigma^*$), then $w = a^{2n}b^{3n}$ for some $n \geq 0$.

Base Case ($k = 1$): The only string of terminals derivable in a single step is via the rule $S \rightarrow \varepsilon$. Thus, $S \Rightarrow \varepsilon$. The empty string can be represented as $a^{2(0)}b^{3(0)}$, satisfying the condition with $n = 0 \geq 0$.

Inductive Hypothesis: Assume that any terminal string derived in m steps, where $m \leq k$, is of the form $a^{2n}b^{3n}$ for some integer $n \geq 0$.

Inductive Step: Consider a terminal string w derived in $k + 1$ steps. The first step of the derivation must utilize the rule $S \rightarrow aaSbbb$. Therefore, the derivation is of the form:

$$S \Rightarrow aaSbbb \Rightarrow^k aaxbbb = w$$

The derivation from S to x takes exactly k steps. By the inductive hypothesis, $x = a^{2m}b^{3m}$ for some $m \geq 0$. Substituting x yields:

$$w = aa(a^{2m}b^{3m})bbb = a^{2(m+1)}b^{3(m+1)}$$

Let $n = m + 1$. Since $m \geq 0$, it follows that $n \geq 1$. Thus, $w = a^{2n}b^{3n}$ for $n \geq 0$.

Part 2: $\{a^{2n}b^{3n} \mid n \geq 0\} \subseteq L(G)$

We proceed by mathematical induction on n to show that $S \Rightarrow^* a^{2n}b^{3n}$ for all $n \geq 0$.

Base Case ($n = 0$): We must show that $S \Rightarrow^* \varepsilon$. Applying the rule $S \rightarrow \varepsilon$ directly achieves this in one step.

Inductive Hypothesis: Assume that for an arbitrary integer $k \geq 0$, $S \Rightarrow^* a^{2k}b^{3k}$.

Inductive Step: We must demonstrate that $S \Rightarrow^* a^{2(k+1)}b^{3(k+1)}$. Beginning with the rule $S \rightarrow aaSbbb$, we have:

$$S \Rightarrow aaSbbb$$

By the inductive hypothesis, the non-terminal S derives $a^{2k}b^{3k}$. Substitution yields:

$$S \Rightarrow aaSbbb \Rightarrow^* aa(a^{2k}b^{3k})bbb = a^{2k+2}b^{3k+3} = a^{2(k+1)}b^{3(k+1)}$$

This completes the inductive step.

Since both $L(G) \subseteq \{a^{2n}b^{3n} \mid n \geq 0\}$ and $\{a^{2n}b^{3n} \mid n \geq 0\} \subseteq L(G)$ hold, we conclude that $L(G) = \{a^{2n}b^{3n} \mid n \geq 0\}$.

Proposed Marking Scheme:

- Correctly identifying the language generated by the grammar: 2 Marks

For the inductive proofs in both directions:

- Clearly stating the base case: 1 Mark
- Correctly stating the inductive hypothesis: 1 Mark

- Correctly executing the inductive step: 2 Marks

3. Let $B = \{w \in \{0,1\}^* \mid \text{the length of } w \text{ is odd and the middle symbol is } 1\}$. Prove or disprove that B is regular. (9 Marks)

Solution: We proceed by contradiction using the Pumping Lemma for regular languages.

Assume, for the sake of contradiction, that B is regular. Then, there exists a pumping length $p \geq 1$ such that any string $s \in B$ of length $|s| \geq p$ can be divided into three parts, $s = xyz$, satisfying the following three conditions:

- (a) $xy^iz \in B$ for all $i \geq 0$
- (b) $|y| > 0$
- (c) $|xy| \leq p$

Consider the string $s = 0^p 1 0^p$. The length of s is $2p + 1$, which is odd and strictly greater than p . The string has exactly p characters before the 1 and p characters after the 1, making 1 the exact middle symbol. Therefore, $s \in B$.

By condition 3 ($|xy| \leq p$), the substring xy must be contained entirely within the first p characters of s . Since the first p characters consist exclusively of 0s, the substring y must consist strictly of 0s. Let $y = 0^k$ for some integer k . By condition 2 ($|y| > 0$), we have $k \geq 1$.

We now “pump down” by choosing $i = 0$. According to condition 1, the resulting string $xy^0z = xz$ must be in B .

Removing y from our original string yields:

$$xz = 0^{p-k} 1 0^p$$

Clearly, the string does not belong to B as the middle symbol cannot be 1 unless $k = 0$. But setting $k = 0$ violates the conditions for pumping lemma.

Proposed Marking Scheme:

- Correctly stating that the statement is false: 1 Mark
 - For stating use of pumping lemma and pumping length p : 1 Mark
 - For taking a valid string s in B with length at least p : 1 Mark
 - For partitioning the string s into xyz and correctly identifying the properties of y : 3 Marks
 - For applying pumping lemma correctly and demonstrating the contradiction: 3 Marks
4. Consider the following two languages E_{DFA} and K_{DFA} defined as follows.
- $E_{\text{DFA}} = \{\langle M \rangle \mid M \text{ is an NFA and } L(M) = \emptyset\}$, and
 - $K_{\text{DFA}} = \{\langle M_1, M_2 \rangle \mid M_1 \text{ and } M_2 \text{ are two DFAs such that } L(M_2) \subseteq L(M_1)\}$.
- (a) Design a procedure that takes an instance $\langle M \rangle$ of E_{DFA} and outputs an instance $\langle M_1, M_2 \rangle$ of K_{DFA} .
 - (b) Formally, prove that the procedure described in part-(a) creates an equivalent instance; i.e. $\langle M \rangle \in E_{\text{DFA}}$ if and only if $\langle M_1, M_2 \rangle \in K_{\text{DFA}}$.

(4 Marks + 6 Marks)

Solution 1:

Consider the following mapping reduction from E_{DFA} to K_{DFA} .

“On input $\langle M \rangle$, where M is an NFA:

- (a) Construct a DFA M_1 explicitly as follows: M_1 has a single state q_0 over alphabet $\{0,1\}$, with transition function $\delta(q_0,0) = \delta(q_0,1) = q_0$. The state q_0 is the (only) start state and there are **no** accepting states. Therefore $L(M_1) = \emptyset$.
- (b) Construct a DFA M_2 equivalent to the NFA M using the standard subset construction algorithm, such that $L(M_2) = L(M)$.
- (c) Output the description $\langle M_1, M_2 \rangle$."

To formally prove the correctness of this reduction, we must show that $\langle M \rangle \in E_{DFA}$ if and only if $\langle M_1, M_2 \rangle \in K_{DFA}$. We demonstrate this in both directions.

Forward Direction ($\Rightarrow$):

Assume $\langle M \rangle \in E_{DFA}$. By the definition of E_{DFA} , M is an NFA and $L(M) = \emptyset$.

According to the construction in step 2 of our procedure, M_2 is a DFA equivalent to M , which means $L(M_2) = L(M) = \emptyset$.

In step 1, we explicitly constructed the DFA M_1 such that $L(M_1) = \emptyset$.

Because the empty set is trivially a subset of itself ($\emptyset \subseteq \emptyset$), it follows that $L(M_2) \subseteq L(M_1)$. Since M_1 and M_2 are both DFAs satisfying the subset condition, $\langle M_1, M_2 \rangle \in K_{DFA}$.

Reverse Direction ($\Leftarrow$):

Assume $\langle M_1, M_2 \rangle \in K_{DFA}$. By the definition of K_{DFA} , M_1 and M_2 are DFAs satisfying the condition $L(M_2) \subseteq L(M_1)$.

From step 1 of our construction, we established that $L(M_1) = \emptyset$.

Therefore, the condition becomes $L(M_2) \subseteq \emptyset$. The only subset of the empty set is the empty set itself, which dictates that $L(M_2) = \emptyset$.

From step 2 of our construction, we know that M_2 was built to be equivalent to the original NFA M , meaning $L(M) = L(M_2) = \emptyset$.

Since M is an NFA whose language is empty, $\langle M \rangle \in E_{DFA}$.

Alternative proof of the reverse direction by contrapositive:

Assume $\langle M \rangle \notin E_{DFA}$, i.e., $L(M) \neq \emptyset$.

Then M_2 (constructed via subset construction from M) satisfies $L(M_2) = L(M) \neq \emptyset$.

But $L(M_1) = \emptyset$ by construction. Since $L(M_2) \neq \emptyset = L(M_1)$, there exists some string $w \in L(M_2)$ with $w \notin L(M_1)$, so $L(M_2) \not\subseteq L(M_1)$, which means $\langle M_1, M_2 \rangle \notin K_{DFA}$. ■

Conclusion:

Since both directions hold, $\langle M \rangle \in E_{DFA} \iff \langle M_1, M_2 \rangle \in K_{DFA}$. The procedure correctly translates an instance of E_{DFA} to an equivalent instance of K_{DFA} . ■

Solution 2 (using decidability of E_{DFA}):

Since E_{DFA} is decidable (the emptiness problem for NFAs is decidable by checking reachability of accepting states), let T_1 be a decider for E_{DFA} .

Consider the following mapping reduction from E_{DFA} to K_{DFA} .

"On input $\langle M \rangle$, where M is an NFA:

- (a) Run the E_{DFA} decider T_1 on $\langle M \rangle$.
- (b) **If T_1 accepts** (i.e., $L(M) = \emptyset$):
 - Convert M to an equivalent DFA M' via subset construction so that $L(M') = L(M) = \emptyset$.
 - Output $\langle M', M' \rangle$.
- (c) **If T_1 rejects** (i.e., $L(M) \neq \emptyset$):
 - Construct a DFA M_1 with a single non-accepting start state q_0 such that $\delta(q_0,0) = \delta(q_0,1) = q_0$, so $L(M_1) = \emptyset$.
 - Convert M to an equivalent DFA M_2 via subset construction so that $L(M_2) = L(M)$.
 - Output $\langle M_1, M_2 \rangle$."

Correctness of Solution 2:

Forward Direction ($\Rightarrow$):

Assume $\langle M \rangle \in E_{\text{DFA}}$, so $L(M) = \emptyset$.

Then T_1 accepts, and we output $\langle M', M' \rangle$ where $L(M') = \emptyset$.

Since $L(M') \subseteq L(M')$ trivially, $\langle M', M' \rangle \in K_{\text{DFA}}$.

Reverse Direction ($\Leftarrow$):

Assume $\langle M \rangle \notin E_{\text{DFA}}$, so $L(M) \neq \emptyset$.

Then T_1 rejects, and we output $\langle M_1, M_2 \rangle$ where $L(M_1) = \emptyset$ and $L(M_2) = L(M) \neq \emptyset$.

Since $L(M_2) \neq \emptyset = L(M_1)$, we have $L(M_2) \not\subseteq L(M_1)$, so $\langle M_1, M_2 \rangle \notin K_{\text{DFA}}$.

In both branches the iff holds, confirming the reduction is correct. ■

Proposed Marking Scheme:

Solution 1 (Mapping reduction via explicit construction):

For part (a):

- For clearly stating the input and output for the reduction: **1 Mark**
- For correctly constructing M_1 to recognize the empty language, with **explicit** description of the single-state DFA (single non-accepting start state q_0 , transitions $\delta(q_0, 0) = \delta(q_0, 1) = q_0$, no final states): **1 Mark**
(Award 0.5 if student only says “DFA with no accepting states” without explicitly describing the single-state construction.)
- For converting the NFA M to an equivalent DFA M_2 via subset construction (deduct 0.5 for not mentioning subset construction): **1 Mark**
- For correctly proving the equivalence of the instances, i.e., $L(M) = L(M_2)$: **1 Mark**

For part (b):

- For proving the forward direction ($\Rightarrow$): **3 Marks**
- For proving the reverse direction ($\Leftarrow$): **3 Marks**
(Full marks also awarded if the student proves the reverse direction by contrapositive.)

Solution 2 (Using decidability of E_{DFA}):

For part (a):

- For clearly stating the input and output for the reduction, and correctly invoking the E_{DFA} decider T_1 : **1 Mark**
- For correctly handling the **accepting branch** of T_1 : outputting $\langle M', M' \rangle$ where M' is the DFA equivalent of M via subset construction: **1 Mark**
- For correctly handling the **rejecting branch** of T_1 : explicitly constructing M_1 (single non-accepting state, $\delta(q_0, 0) = \delta(q_0, 1) = q_0$) and M_2 via subset construction from M : **1 Mark**
- For correctly outputting $\langle M_1, M_2 \rangle$: **1 Mark**

For part (b):

- For proving the forward direction ($\Rightarrow$) correctly for both branches of T_1 : **3 Marks**
 - For proving the reverse direction ($\Leftarrow$) correctly for both branches, or equivalently by contrapositive: **3 Marks**
-

5. Consider the language $RL_{TM} = \{\langle M \rangle \mid M \text{ is a Turing Machine such that } L(M) \text{ contains all strings over } \{0,1\}^* \text{ that start with 0 and end with 1}\}$. Additionally, suppose that $A_{TM} = \{\langle M, w \rangle \mid M \text{ is a Turing Machine and } M \text{ accepts } w\}$.

- (a) Design a reduction procedure that takes an input instance $\langle M, w \rangle$ of A_{TM} and outputs an instance $\langle \widehat{M} \rangle$ of RL_{TM} .
- (b) Formally prove that your reduction procedure produces an equivalent instance.

(4 Marks + 6 Marks)

Solution: Consider the following mapping reduction from A_{TM} to RL_{TM} . “On input $\langle M, w \rangle$, where M is a Turing machine and w is a string:

- (a) Construct the following Turing machine $\widehat{M}$:
 $\widehat{M} = \text{‘On input } x\text{:}$
 i. Ignore the input x .
 ii. Simulate M on input w .
 iii. If M accepts w , **accept**. If M rejects w , **reject**.’
- (b) Output the description $\langle \widehat{M} \rangle$.”

To prove the correctness of this reduction, we must show that $\langle M, w \rangle \in A_{TM}$ if and only if $F(\langle M, w \rangle) = \langle \widehat{M} \rangle \in RL_{TM}$.

Forward Direction ($\Rightarrow$):

Assume $\langle M, w \rangle \in A_{TM}$. By definition, this means the Turing machine M accepts the string w . Consider the behavior of the constructed machine $\widehat{M}$. On any input string x , $\widehat{M}$ simulates M on w . Since M accepts w , $\widehat{M}$ will eventually halt and accept x . Because this holds true for any arbitrary string x , the language of $\widehat{M}$ is the set of all strings over $\{0,1\}^*$, i.e., $L(\widehat{M}) = \{0,1\}^*$. Since $\{0,1\}^*$ trivially contains all strings over $\{0,1\}^*$ that start with 0 and end with 1, we conclude that $\langle \widehat{M} \rangle \in RL_{TM}$.

Reverse Direction ($\Leftarrow$) [Proof by Contrapositive]:

We prove the contrapositive: If $\langle M, w \rangle \notin A_{TM}$, then $\langle \widehat{M} \rangle \notin RL_{TM}$.

Assume $\langle M, w \rangle \notin A_{TM}$. By definition, this means M either rejects w or loops infinitely on w . In either case, M does not accept w .

Since M does not accept w , the simulation of M on w within $\widehat{M}$ will never reach an accept state. Consequently, $\widehat{M}$ will not accept any input x .

Thus, $L(\widehat{M}) = \emptyset$. Since the empty set does not contain any strings, it certainly does not contain the strings starting with 0 and ending with 1. Therefore, $\langle \widehat{M} \rangle \notin RL_{TM}$.

By contrapositive, we conclude that if $\langle \widehat{M} \rangle \in RL_{TM}$, then $\langle M, w \rangle \in A_{TM}$.

We have shown that $\langle M, w \rangle \in A_{TM} \iff \langle \widehat{M} \rangle \in RL_{TM}$. ■

Proposed Marking Scheme:

For part (a):

- For clearly stating the input and output for the reduction: 1 Mark
- For ignoring the input x in the construction of $\widehat{M}$: 1 Mark
- For correctly simulating M on w within $\widehat{M}$: 2 Marks

For part (b):

- For proving the forward direction ($\Rightarrow$): 3 Marks
- For proving the reverse direction ($\Leftarrow$): 3 Marks

Deduct marks for any logical errors, incorrect assumptions, or incomplete arguments in the proofs.

6. Let $B = \{0^n \mid n \text{ is a prime}\}$. Prove or disprove that B is a context-free language.

Solution: We proceed by contradiction using the Pumping Lemma for context-free languages.

Assume, for the sake of contradiction, that B is a context-free language. Let $p \geq 1$ be the pumping length given by the Pumping Lemma.

Choose a prime number $q \geq p$. Let our string be $s = 0^q$. Clearly, $s \in B$ because its length q is prime, and $|s| = q \geq p$.

By the Pumping Lemma, s can be divided into five parts, $s = uvxyz$, satisfying the following three conditions:

- (a) $uv^i xy^i z \in B$ for all $i \geq 0$
- (b) $|vy| > 0$
- (c) $|vxy| \leq p$

Because the alphabet consists of a single symbol, the substrings u, v, x, y , and z all consist exclusively of 0s. Let $k = |vy|$ represent the combined length of the pumped parts. By condition 2, we know that $k \geq 1$.

We now “pump up” the string by choosing $i = q + 1$.

According to condition 1, the pumped string $s' = uv^{q+1}xy^{q+1}z$ must be in B . Let us calculate the length of this new string s' :

$$\begin{aligned} |s'| &= |s| + (i - 1)|vy| \\ &= q + ((q + 1) - 1) \cdot k \\ &= q + qk \\ &= q(1 + k) \end{aligned}$$

Since q is a prime number, we know $q \geq 2$. Furthermore, since condition 2 guarantees $k \geq 1$, we have $1 + k \geq 2$.

Therefore, the length $|s'| = q(1 + k)$ is the product of two integers strictly greater than 1. This means the length of s' is a composite number.

Because the length of s' is not prime, $s' \notin B$. This directly contradicts the first condition of the Pumping Lemma, which states that all pumped strings must remain in B .

Therefore, our initial assumption must be false, and B is not a context-free language.

Proposed Marking Scheme:

- Correctly stating that the statement is false: 1 Mark
 - For stating use of pumping lemma and pumping length p : 1 Mark
 - For taking a valid string s in B with length at least p : 1 Mark
 - For partitioning the string s into $uvxyz$ and correctly identifying the properties of v and y : 2 Marks
 - For applying pumping lemma correctly and demonstrating the contradiction: 3 Marks
7. Let L be an undecidable language and $L \leq_m A_{TM}$ (i.e. there exists a mapping reduction from L to A_{TM}).

Then, prove or disprove that $\bar{L} \leq_m A_{TM}$. (8 Marks)

Hint: Use closure properties and any related property.

Solution: We know that the language

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ is a Turing machine that accepts } w\}$$

is Turing Recognizable but not decidable. Since $L \leq_m A_{TM}$, we can conclude that L is Turing Recognizable. We now claim that $\bar{L}$ is not Turing-recognizable. Assume, for the sake of contradiction, that $\bar{L}$ is Turing-recognizable. Then, L is Turing-recognizable, and co-Turing-recognizable. This would imply that L is decidable, which contradicts the given fact that L is undecidable.

Finally, assume $\bar{L} \leq_m A_{TM}$. This would imply that $\bar{L}$ is Turing-recognizable, leading to a contradiction. Hence, we can conclude that $\bar{L} \not\leq_m A_{TM}$. ■

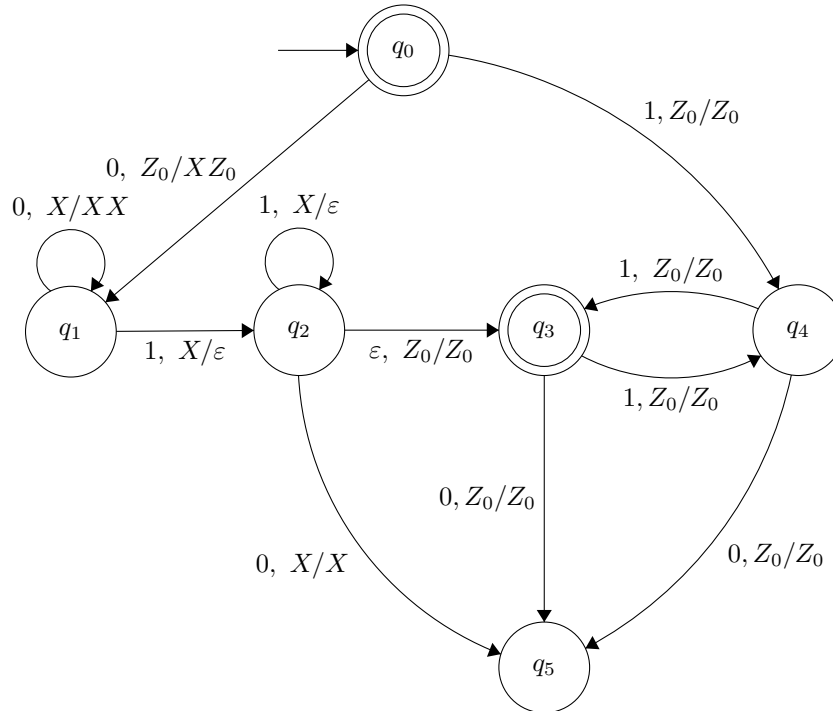
Proposed Marking Scheme:

- For correctly stating disprove. 1 Mark
- For correctly stating that A_{TM} is Turing Recognizable but not decidable: 1 Mark
- For correctly concluding that L is Turing Recognizable: 1 Mark
- For correctly claiming that $\bar{L}$ is not Turing Recognizable: 1 Mark
- For contradicting the undecidability of L : 2 Marks
- Finally proving via contradiction that $L \not\leq_m A_{TM}$: 2 Marks

8. Let $B = \{0^n 1^{m+n} 1^m \mid m, n \geq 0\}$. Design a deterministic pushdown automata for B . No proof is required. Transition diagram is sufficient. But you must specify whether you are accepting with final state or empty stack, and you must specify the stack symbols explicitly.

(7 Marks)

Solution:



Proposed Marking Scheme:

- 1 Mark: For correctly stating the stack symbols, particularly specifying which stack symbol is used for the push and pop operations of the input alphabet.

- 1 Mark: For correctly stating the stack acceptance condition (final state or empty stack).
- 5 Marks: For the correctness of the transition diagram. Deduct marks for any of the following:
 - Missing or incorrect transitions
 - Unreachable states
 - Violations of determinism (i.e., for each state and input symbol, including ε , there must be at most one possible transition)